

EDA

1. Filtered View

```
/* =====  
SuperStore Sales SQL Project  
EDA  
Author: Asiya Nizam  
Tool: pgAdmin (PostgreSQL)  
=====*/
```

```
/* =====  
Create Filtered View  
Purpose: Restrict analysis to 2023 - 2025 business data  
=====*/
```

```
CREATE OR REPLACE VIEW orders_2023_2025 AS  
SELECT *  
FROM orders  
WHERE order_date < DATE '2026-01-01';
```

Output:

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 388 msec.

The screenshot displays the pgAdmin 4 interface. On the left, the 'Object Explorer' shows the database structure, including the 'orders_2023_2025' view. The main pane shows the 'Data Output' tab for the query 'SELECT * FROM public.orders_2023_2025 LIMIT 100'. The query results are displayed in a table with 16 rows and 10 columns: row_id, order_id, order_date, ship_date, ship_mode, customer_id, customer_name, segment, and country. The data shows various orders from 2023, including Standard Class, First Class, and Second Class shipments.

row_id	order_id	order_date	ship_date	ship_mode	customer_id	customer_name	segment	country
1	US-2023-103800	2023-01-03	2023-01-07	Standard Class	DP-13000	Darren Powers	Consumer	United St
2	US-2023-112326	2023-01-04	2023-01-08	Standard Class	PO-19195	Phillina Ober	Home Office	United St
3	US-2023-112326	2023-01-04	2023-01-08	Standard Class	PO-19195	Phillina Ober	Home Office	United St
4	US-2023-112326	2023-01-04	2023-01-08	Standard Class	PO-19195	Phillina Ober	Home Office	United St
5	US-2023-141817	2023-01-05	2023-01-12	Standard Class	MB-18085	Mick Brown	Consumer	United St
6	US-2023-167199	2023-01-06	2023-01-10	Standard Class	ME-17320	Maria Etezadi	Home Office	United St
7	US-2023-167199	2023-01-06	2023-01-10	Standard Class	ME-17320	Maria Etezadi	Home Office	United St
8	US-2023-106054	2023-01-06	2023-01-07	First Class	JO-15145	Jack O'Briant	Corporate	United St
9	US-2023-167199	2023-01-06	2023-01-10	Standard Class	ME-17320	Maria Etezadi	Home Office	United St
10	US-2023-167199	2023-01-06	2023-01-10	Standard Class	ME-17320	Maria Etezadi	Home Office	United St
11	US-2023-167199	2023-01-06	2023-01-10	Standard Class	ME-17320	Maria Etezadi	Home Office	United St
12	US-2023-130813	2023-01-06	2023-01-08	Second Class	LS-17230	Lycoris Saunders	Consumer	United St
13	US-2023-167199	2023-01-06	2023-01-10	Standard Class	ME-17320	Maria Etezadi	Home Office	United St
14	US-2023-167199	2023-01-06	2023-01-10	Standard Class	ME-17320	Maria Etezadi	Home Office	United St
15	US-2023-105417	2023-01-07	2023-01-12	Standard Class	VS-21820	Vivek Sundaresam	Consumer	United St
16	US-2023-105417	2023-01-07	2023-01-12	Standard Class	VS-21820	Vivek Sundaresam	Consumer	United St

2. Filtered Dataset Check

```
/* =====
Filtered Dataset Check
=====*/

SELECT *
FROM orders_2023_2025
LIMIT 10;
```

Output:

Data Output

Messages

Notifications

3. Business KPI (Total Sales & Profit)

```
/* =====
Business KPI (Total Sales & Profit)
=====*/

SELECT
  ROUND(SUM(sales),2) AS total_sales,
  ROUND(SUM(profit),2) AS total_profit,
  ROUND(
    (SUM(profit)/SUM(sales)) * 100,
    2
  ) AS profit_margin_pct,
  COUNT(DISTINCT order_id) AS total_orders,
  COUNT(DISTINCT customer_id) AS total_customers
FROM orders_2023_2025;
```

Output:

	total_sales numeric	total_profit numeric	profit_margin_pct numeric	total_orders bigint	total_customers bigint
1	1580967.08	196370.72	12.42	3388	793

4. Yearly Sales Trend

```

/* =====
Yearly Sales Trend
=====*/

WITH yearly_sales_trend AS (

    SELECT
        EXTRACT(YEAR FROM order_date) AS year,
        ROUND(SUM(sales),2) AS yearly_sales,
        ROUND(SUM(profit),2) AS yearly_profit
    FROM orders_2023_2025
    GROUP BY year

)

SELECT year,
       yearly_sales,
       yearly_profit,
       LAG(yearly_sales) OVER(ORDER BY year) AS previous_year_sales,
       ROUND(
           (
               yearly_sales -
               LAG(yearly_sales) OVER(ORDER BY year)
           ) * 100
           /
           LAG(yearly_sales) OVER(ORDER BY year),
           2
       ) AS sales_growth_pct
FROM yearly_sales_trend
ORDER BY year;

```

Output:

	year numeric 🔒	yearly_sales numeric 🔒	yearly_profit numeric 🔒	previous_year_sales numeric 🔒	sales_growth_pct numeric 🔒
1	2023	494040.27	51684.38	[null]	[null]
2	2024	472992.98	62021.03	494040.27	-4.26
3	2025	613933.83	82665.31	472992.98	29.80

5. Monthly Sales Trend

```

/* =====
Monthly Sales Trend
=====*/

WITH monthly_sales_trend AS (

    SELECT
        EXTRACT(YEAR FROM order_date) AS year,
        TO_CHAR(order_date, 'Mon') AS month_name,
        DATE_TRUNC('Month', order_date) AS month_date,
        ROUND(SUM(sales),2) AS monthly_revenue
    FROM orders_2023_2025
    GROUP BY
        year,
        month_name,
        month_date

)

SELECT

```

```

SELECT
    year,
    month_name,
    month_date,
    monthly_revenue,
    LAG(monthly_revenue) OVER(ORDER BY month_date) AS previous_month_revenue,
    ROUND(
        (
            monthly_revenue -
            LAG(monthly_revenue)
            OVER(ORDER BY month_date)
        ) * 100
        /
        LAG(monthly_revenue)
        OVER(ORDER BY month_date),
        2
    ) AS sales_growth_pct
FROM monthly_sales_trend
ORDER BY
    month_date;

```

Output:

Data Output Messages Notifications							
Showing rows: 1 to 36							
	year numeric	month_name text	month_date timestamp with time zone	monthly_revenue numeric	previous_month_revenue numeric	sales_growth_pct numeric	
1	2023	Jan	2023-01-01 00:00:00+00	14518.06	[null]	[null]	
2	2023	Feb	2023-02-01 00:00:00+00	4519.92	14518.06	-68.87	
3	2023	Mar	2023-03-01 00:00:00+00	56933.94	4519.92	1159.62	
4	2023	Apr	2023-04-01 00:00:00+01	28295.35	56933.94	-50.30	
5	2023	May	2023-05-01 00:00:00+01	26319.76	28295.35	-6.98	
6	2023	Jun	2023-06-01 00:00:00+01	34669.49	26319.76	31.72	
7	2023	Jul	2023-07-01 00:00:00+01	33946.37	34669.49	-2.09	
8	2023	Aug	2023-08-01 00:00:00+01	28918.34	33946.37	-14.81	
9	2023	Sep	2023-09-01 00:00:00+01	82670.42	28918.34	185.88	
10	2023	Oct	2023-10-01 00:00:00+01	32413.32	82670.42	-60.79	
11	2023	Nov	2023-11-01 00:00:00+00	78826.98	32413.32	143.19	
12	2023	Dec	2023-12-01 00:00:00+00	72008.32	78826.98	-8.65	
13	2024	Jan	2024-01-01 00:00:00+00	18461.92	72008.32	-74.36	
14	2024	Feb	2024-02-01 00:00:00+00	11951.40	18461.92	-35.26	
15	2024	Mar	2024-03-01 00:00:00+00	39978.64	11951.40	234.51	
16	2024	Apr	2024-04-01 00:00:00+01	34195.25	39978.64	-14.47	
17	2024	May	2024-05-01 00:00:00+01	30246.98	34195.25	-11.55	
18	2024	Jun	2024-06-01 00:00:00+01	24797.31	30246.98	-18.02	
19	2024	Jul	2024-07-01 00:00:00+01	28765.32	24797.31	16.00	
20	2024	Aug	2024-08-01 00:00:00+01	37056.99	28765.32	28.83	
21	2024	Sep	2024-09-01 00:00:00+01	64627.00	37056.99	74.40	
Total rows: 36		Query complete 00:00:00.128					

6. Sales by Region

```

/* =====
Sales By Region
=====*/

SELECT
    region,
    COUNT(DISTINCT order_id) AS total_orders,
    ROUND(SUM(sales),2) AS total_sales,
    ROUND(SUM(profit),2) AS total_profit,
    ROUND(
        (SUM(profit)/SUM(sales))*100
    ,2) AS profit_margin_pct,
    ROUND(SUM(sales)/COUNT(DISTINCT order_id),2) AS avg_order_value
FROM orders_2023_2025
GROUP BY region
ORDER BY total_sales DESC;

```

Output:

	region character varying (50)	total_orders bigint	total_sales numeric	total_profit numeric	profit_margin_pct numeric	avg_order_value numeric
1	West	1089	481572.69	65373.99	13.58	442.22
2	East	979	475261.18	60802.86	12.79	485.46
3	Central	771	355317.13	32293.14	9.09	460.85
4	South	549	268816.08	37900.73	14.10	489.65

7. Top 10 States by Sales in each Country

```

/* =====
Top 10 States by Sales in Each Country
=====*/

WITH states_ranked AS (

    SELECT
        country_region,
        state_province,
        COUNT(DISTINCT order_id) AS total_orders,
        ROUND(SUM(sales),2) AS total_sales,
        ROUND(SUM(profit),2) AS total_profit,
        DENSE_RANK() OVER(PARTITION BY country_region ORDER BY SUM(sales) DESC) AS rank
    FROM orders_2023_2025
    GROUP BY
        country_region,
        state_province

)

SELECT *
FROM states_ranked
WHERE rank <= 10
ORDER BY
    country_region,
    rank;

```

Output:

	country_region character varying (100)	state_province character varying (100)	total_orders bigint	total_sales numeric	total_profit numeric	rank bigint
1	Canada	Ontario	13	5409.96	1509.04	1
2	Canada	Alberta	6	4610.31	624.84	2
3	Canada	Quebec	21	2873.21	827.82	3
4	Canada	British Columbia	9	1568.84	119.68	4
5	Canada	Manitoba	2	1175.31	137.60	5
6	Canada	Prince Edward Island	2	543.95	164.68	6
7	Canada	Newfoundland and Labra...	3	316.96	115.83	7
8	Canada	Nova Scotia	4	249.43	-51.16	8
9	Canada	New Brunswick	5	169.26	-55.62	9
10	Canada	Saskatchewan	1	63.97	19.99	10
11	United States	California	677	311299.35	47014.92	1
12	United States	New York	388	216953.22	49681.59	2
13	United States	Texas	323	126766.27	-16890.81	3
14	United States	Pennsylvania	189	73823.65	-10447.20	4
15	United States	Washington	160	73101.36	16145.84	5
16	United States	Virginia	91	63036.52	16791.98	6
17	United States	Florida	130	63029.01	-3643.38	7
18	United States	Illinois	176	55814.56	-5862.34	8
19	United States	Ohio	150	54993.35	-15234.85	9
20	United States	Michigan	83	50435.97	15975.41	10
Total rows: 20		Query complete 00:00:00.095				

8. Sales by Category

```

/* =====
Sales by Category
=====*/

```

```

SELECT
    category,
    ROUND(SUM(sales),2) AS total_sales,
    ROUND(SUM(profit),2) AS total_profit,
    ROUND(
        (SUM(profit)/SUM(sales))*100
    ,2) AS profit_margin_pct
FROM orders_2023_2025
GROUP BY category
ORDER BY total_sales DESC;

```

Output:

	category character varying (50)	total_sales numeric	total_profit numeric	profit_margin_pct numeric
1	Technology	567474.17	95709.62	16.87
2	Furniture	535435.01	16380.66	3.06
3	Office Supplies	478057.90	84280.44	17.63

9. Sales Performance by Subcategory within each Category

```

/* =====
Sales Performance by Subcategory within each Category
(Using subquery instead of CTE)
=====*/

SELECT *
FROM
(
    SELECT
        category,
        sub_category,
        ROUND(SUM(sales),2) AS total_sales,
        ROUND(SUM(profit),2) AS total_profit,
        DENSE_RANK() OVER(PARTITION BY category ORDER BY SUM(sales) DESC) AS rank
    FROM orders_2023_2025
    GROUP BY
        category,
        sub_category
) AS subcategory_ranked
ORDER BY
    category,
    rank;

```

Output:

	category character varying (50) 🔒	sub_category character varying (50) 🔒	total_sales numeric 🔒	total_profit numeric 🔒	rank bigint 🔒
1	Furniture	Chairs	237735.92	19435.42	1
2	Furniture	Tables	147021.09	-9661.03	2
3	Furniture	Bookcases	85007.17	-2973.88	3
4	Furniture	Furnishings	65670.83	9580.15	4
5	Office Supplies	Storage	154370.75	13921.85	1
6	Office Supplies	Binders	133703.61	23485.07	2
7	Office Supplies	Appliances	65102.05	10414.56	3
8	Office Supplies	Paper	51367.08	22257.30	4
9	Office Supplies	Supplies	30649.83	-227.02	5
10	Office Supplies	Art	18636.32	4382.51	6
11	Office Supplies	Envelopes	13149.75	5546.21	7
12	Office Supplies	Labels	8708.01	3770.20	8
13	Office Supplies	Fasteners	2370.50	729.76	9
14	Technology	Phones	226174.24	32098.66	1
15	Technology	Machines	146037.20	6292.55	2
16	Technology	Accessories	107434.10	26264.32	3
17	Technology	Copiers	87828.63	31054.09	4

10. Top 10 Customers by Sales

```

/* =====
Top 10 Customers by Sales
=====*/

SELECT
    customer_name,
    COUNT(DISTINCT order_id) AS total_orders,
    ROUND(SUM(sales),2) AS total_sales,
    ROUND(SUM(profit),2) AS total_profit,
    ROUND(SUM(sales)/COUNT(DISTINCT order_id),2) AS avg_order_value
FROM orders_2023_2025
GROUP BY customer_name
ORDER BY total_sales DESC
LIMIT 10;

```

Output:

	customer_name character varying (100) 🔒	total_orders bigint 🔒	total_sales numeric 🔒	total_profit numeric 🔒	avg_order_value numeric 🔒
1	Sean Miller	3	24508.65	-2015.03	8169.55
2	Tamara Chand	5	19052.22	8981.32	3810.44
3	Sanjit Chand	8	14129.64	5751.58	1766.21
4	Ken Lonsdale	9	13604.84	645.12	1511.65
5	Adrian Barton	5	12182.53	5103.67	2436.51
6	Christopher Conant	3	11901.19	2083.34	3967.06
7	Becky Martin	3	11432.85	-1759.87	3810.95
8	Sanjit Engle	7	10663.68	2887.50	1523.38
9	Greg Tran	7	10623.28	2079.82	1517.61
10	Bill Shonely	5	10501.65	2616.06	2100.33

11. Sales by Customer Segment

```

/* =====
Sales by Customer Segment
=====*/

SELECT
    segment,
    COUNT(DISTINCT customer_id) AS total_customers,
    COUNT(DISTINCT order_id) AS total_orders,
    ROUND(SUM(sales),2) AS total_sales,
    ROUND(SUM(profit),2) AS total_profit,
    ROUND(
        (SUM(profit)/SUM(sales)) * 100,
        2
    ) AS profit_margin_pct,
    ROUND(SUM(sales)/COUNT(DISTINCT order_id),2) AS avg_order_value
FROM orders_2023_2025
GROUP BY segment
ORDER BY total_sales DESC;

```

Output:

	segment character varying (50)	total_customers bigint	total_orders bigint	total_sales numeric	total_profit numeric	profit_margin_pct numeric	avg_order_value numeric
1	Consumer	409	1738	835747.94	90101.14	10.78	480.87
2	Corporate	236	1044	467562.07	65842.48	14.08	447.86
3	Home Office	148	606	277657.07	40427.10	14.56	458.18

12. Sales by Ship mode

```

/* =====
Sales by Shipmode
=====*/

SELECT
    ship_mode,
    COUNT(DISTINCT order_id) AS total_orders,
    ROUND(SUM(sales),2) AS total_sales,
    ROUND(SUM(profit),2) AS total_profit,
    ROUND(
        AVG(ship_date - order_date)::NUMERIC,
        1
    ) AS avg_days_to_ship
FROM orders_2023_2025
GROUP BY ship_mode
ORDER BY total_sales DESC;

```

Output:

	ship_mode character varying (50)	total_orders bigint	total_sales numeric	total_profit numeric	avg_days_to_ship numeric
1	Standard Class	2052	973042.89	116981.86	5.0
2	Second Class	652	315961.46	37991.35	3.2
3	First Class	508	211545.53	28632.93	2.2
4	Same Day	176	80417.20	12764.58	0.0

13. Impact of Discount on Profitability

```

/* =====
Impact of Discount on Profitability
=====*/

SELECT
    CASE
        WHEN discount = 0 THEN '0% - No Discount'
        WHEN discount <= 0.1 THEN '1 - 10%'
        WHEN discount <= 0.2 THEN '11 - 20%'
        WHEN discount <= 0.3 THEN '21 - 30%'
        WHEN discount <= 0.4 THEN '31 - 40%'
        WHEN discount <= 0.5 THEN '41 - 50%'
        ELSE 'Above 50%'
    END AS discount_band,
    MIN(discount) AS sort_order,
    COUNT(*) AS total_rows,
    ROUND(SUM(sales),2) AS total_sales,
    ROUND(SUM(profit),2) AS total_profit,
    ROUND(AVG(profit),2) AS average_profit
FROM orders_2023_2025
GROUP BY discount_band
ORDER BY sort_order;

```

Output:

	discount_band text	sort_order numeric	total_rows bigint	total_sales numeric	total_profit numeric	average_profit numeric
1	0% - No Discount	0.00	3296	748350.61	220093.62	66.78
2	1 - 10%	0.10	67	35773.73	5796.95	86.52
3	11 - 20%	0.15	2500	540200.64	58682.01	23.47
4	21 - 30%	0.30	161	72640.69	-6593.25	-40.95
5	31 - 40%	0.32	154	93174.08	-17152.16	-111.38
6	41 - 50%	0.45	54	48092.54	-14906.77	-276.05
7	Above 50%	0.60	583	42734.79	-49549.68	-84.99